

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №7

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:

Я.В. Автушенко

Студент группы ПО-13

Проверил:

А.Д. Кулик

преп.-стажёр кафедры ИИТ

Цель: освоить возможности языка программирования Python в разработке оконных приложений

Задание 1: Построение графических примитивов и надписей

Требования к выполнению

- Реализовать соответствующие классы, указанные в задании;
- Организовать ввод параметров для создания объектов (использовать экранные компоненты);
- Осуществить визуализацию графических примитивов

Важное замечание: должна быть предусмотрена возможность приостановки выполнения визуализации, изменения параметров «на лету» и снятия скриншотов с сохранением в текущую активную директорию.

Для всех динамических сцен необходимо задавать параметр скорости!

1) Создать класс Triangle и класс Point. Объявить список из n объектов класса Point, написать функцию, определяющую, какая из точек лежит внутри, а какая – снаружи треугольника.

```
import tkinter as tk
from tkinter import ttk, messagebox
import math
import random
from PIL import ImageGrab
import os
from datetime import datetime
```

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
```

```
return f"Point({self.x:.1f}, {self.y:.1f})"
```

```
class Triangle:
```

```
    def __init__(self, p1, p2, p3):
```

```
        self.p1 = p1
```

```
        self.p2 = p2
```

```
        self.p3 = p3
```

```
    def area(self):
```

```
        return abs((self.p1.x*(self.p2.y - self.p3.y) +
```

```
                    self.p2.x*(self.p3.y - self.p1.y) +
```

```
                    self.p3.x*(self.p1.y - self.p2.y)) / 2.0)
```

```
    def is_point_inside(self, point):
```

```
        # Вычисляем площади подтреугольников
```

```
        t1 = Triangle(point, self.p2, self.p3)
```

```
        t2 = Triangle(self.p1, point, self.p3)
```

```
        t3 = Triangle(self.p1, self.p2, point)
```

```
        total_area = self.area()
```

```
        return abs(total_area - (t1.area() + t2.area() + t3.area())) < 1e-9
```

```
class App:
```

```
    def __init__(self, root):
```

```
        self.root = root
```

```
        self.root.title("Треугольник и точки")
```

```
        self.root.geometry("900x700")
```

```
        self.points = []
```

```
        self.triangle = None
```

```
        self.speed = tk.DoubleVar(value=1.0)
```

```
        self.paused = False
```

```
self.animating = False
```

```
self.setup_ui()
```

```
self.setup_canvas()
```

```
def setup_ui(self):
```

```
    # Панель управления
```

```
    control_frame = ttk.Frame(self.root, padding=10)
```

```
    control_frame.pack(side=tk.TOP, fill=tk.X)
```

```
    # Координаты треугольника
```

```
    ttk.Label(control_frame, text="Треугольник:").grid(row=0, column=0, columnspan=3,  
sticky=tk.W)
```

```
    ttk.Label(control_frame, text="X1,Y1:").grid(row=1, column=0, sticky=tk.W)
```

```
    self.t_p1 = ttk.Entry(control_frame, width=10)
```

```
    self.t_p1.grid(row=1, column=1)
```

```
    self.t_p1.insert(0, "200,100")
```

```
    ttk.Label(control_frame, text="X2,Y2:").grid(row=1, column=2, sticky=tk.W)
```

```
    self.t_p2 = ttk.Entry(control_frame, width=10)
```

```
    self.t_p2.grid(row=1, column=3)
```

```
    self.t_p2.insert(0, "100,400")
```

```
    ttk.Label(control_frame, text="X3,Y3:").grid(row=1, column=4, sticky=tk.W)
```

```
    self.t_p3 = ttk.Entry(control_frame, width=10)
```

```
    self.t_p3.grid(row=1, column=5)
```

```
    self.t_p3.insert(0, "500,300")
```

```
    # Количество точек
```

```
    ttk.Label(control_frame, text="Кол-во точек:").grid(row=2, column=0, sticky=tk.W,  
pady=5)
```

```

self.points_count = ttk.Entry(control_frame, width=10)
self.points_count.grid(row=2, column=1, pady=5)
self.points_count.insert(0, "50")

# Скорость
ttk.Label(control_frame, text="Скорость:").grid(row=2, column=2, sticky=tk.W)
self.speed_scale = ttk.Scale(control_frame, from_=0.1, to=5.0, variable=self.speed,
orient=tk.HORIZONTAL, length=100)
self.speed_scale.grid(row=2, column=3, columnspan=2)
self.speed_label = ttk.Label(control_frame, text="1.0x")
self.speed_label.grid(row=2, column=5)
self.speed.trace_add("write", lambda *args:
self.speed_label.configure(text=f"{self.speed.get():.1f}x"))

# Кнопки
btn_frame = ttk.Frame(control_frame)
btn_frame.grid(row=3, column=0, columnspan=6, pady=10)

ttk.Button(btn_frame, text="Создать/Обновить",
command=self.create_objects).pack(side=tk.LEFT, padx=5)

ttk.Button(btn_frame, text="⏸ Пауза/Продолжить",
command=self.toggle_pause).pack(side=tk.LEFT, padx=5)

ttk.Button(btn_frame, text="📷 Скриншот",
command=self.take_screenshot).pack(side=tk.LEFT, padx=5)

ttk.Button(btn_frame, text="Очистить", command=self.clear_canvas).pack(side=tk.LEFT,
padx=5)

# Статистика
self.stats_label = ttk.Label(control_frame, text="Готов к работе")
self.stats_label.grid(row=4, column=0, columnspan=6, sticky=tk.W)

def setup_canvas(self):
self.canvas = tk.Canvas(self.root, bg='white', width=850, height=500)
self.canvas.pack(side=tk.BOTTOM, fill=tk.BOTH, expand=True, padx=10, pady=10)

```

```
def parse_point(self, entry_str):
```

```
    try:
```

```
        x, y = map(float, entry_str.split(','))
```

```
        return Point(x, y)
```

```
    except:
```

```
        return None
```

```
def create_objects(self):
```

```
    try:
```

```
        # Создаем треугольник
```

```
        p1 = self.parse_point(self.t_p1.get())
```

```
        p2 = self.parse_point(self.t_p2.get())
```

```
        p3 = self.parse_point(self.t_p3.get())
```

```
    if not all([p1, p2, p3]):
```

```
        messagebox.showerror("Ошибка", "Неверные координаты треугольника")
```

```
        return
```

```
    self.triangle = Triangle(p1, p2, p3)
```

```
    # Создаем точки
```

```
    n = int(self.points_count.get())
```

```
    self.points = []
```

```
    for i in range(n):
```

```
        x = random.uniform(50, 800)
```

```
        y = random.uniform(50, 450)
```

```
        self.points.append(Point(x, y))
```

```
    self.draw_scene()
```

```
    self.update_stats()
```

```
except ValueError:
```

```
    messagebox.showerror("Ошибка", "Неверное количество точек")
```

```
def draw_scene(self):
```

```
    self.canvas.delete("all")
```

```
    if not self.triangle:
```

```
        return
```

```
    # Рисуем треугольник
```

```
    self.canvas.create_polygon(
```

```
        self.triangle.p1.x, self.triangle.p1.y,
```

```
        self.triangle.p2.x, self.triangle.p2.y,
```

```
        self.triangle.p3.x, self.triangle.p3.y,
```

```
        outline='blue', fill='lightblue', width=2
```

```
    )
```

```
    # Рисуем точки
```

```
    for point in self.points:
```

```
        if self.triangle.is_point_inside(point):
```

```
            color = 'green'
```

```
            size = 6
```

```
        else:
```

```
            color = 'red'
```

```
            size = 4
```

```
    self.canvas.create_oval(
```

```
        point.x - size, point.y - size,
```

```
        point.x + size, point.y + size,
```

```
        fill=color, outline='black'
```

```
    )
```

```

# Подписываем вершины
for i, p in enumerate([self.triangle.p1, self.triangle.p2, self.triangle.p3]):
    self.canvas.create_text(p.x, p.y - 15, text=f"P {i+1}", fill='blue', font=('Arial', 10, 'bold'))

def update_stats(self):
    if not self.triangle:
        return

    inside = sum(1 for p in self.points if self.triangle.is_point_inside(p))
    outside = len(self.points) - inside

    self.stats_label.configure(
        text=f"Всего точек: {len(self.points)} | Внутри: {inside} (зеленые) | Снаружи: {outside} (красные)"
    )

def toggle_pause(self):
    self.paused = not self.paused

def clear_canvas(self):
    self.canvas.delete("all")
    self.points = []
    self.triangle = None
    self.stats_label.configure(text="Очищено")

def take_screenshot(self):
    # Получаем координаты холста
    x = self.root.winfo_rootx() + self.canvas.winfo_x() + 10
    y = self.root.winfo_rooty() + self.canvas.winfo_y() + 10
    width = self.canvas.winfo_width() - 20
    height = self.canvas.winfo_height() - 20

```



```
# Делаем скриншот
```

```
screenshot = ImageGrab.grab(bbox=(x, y, x + width, y + height))
```

```
# Сохраняем в текущую директорию
```

```
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
```

```
filename = f"screenshot_{timestamp}.png"
```

```
screenshot.save(filename)
```

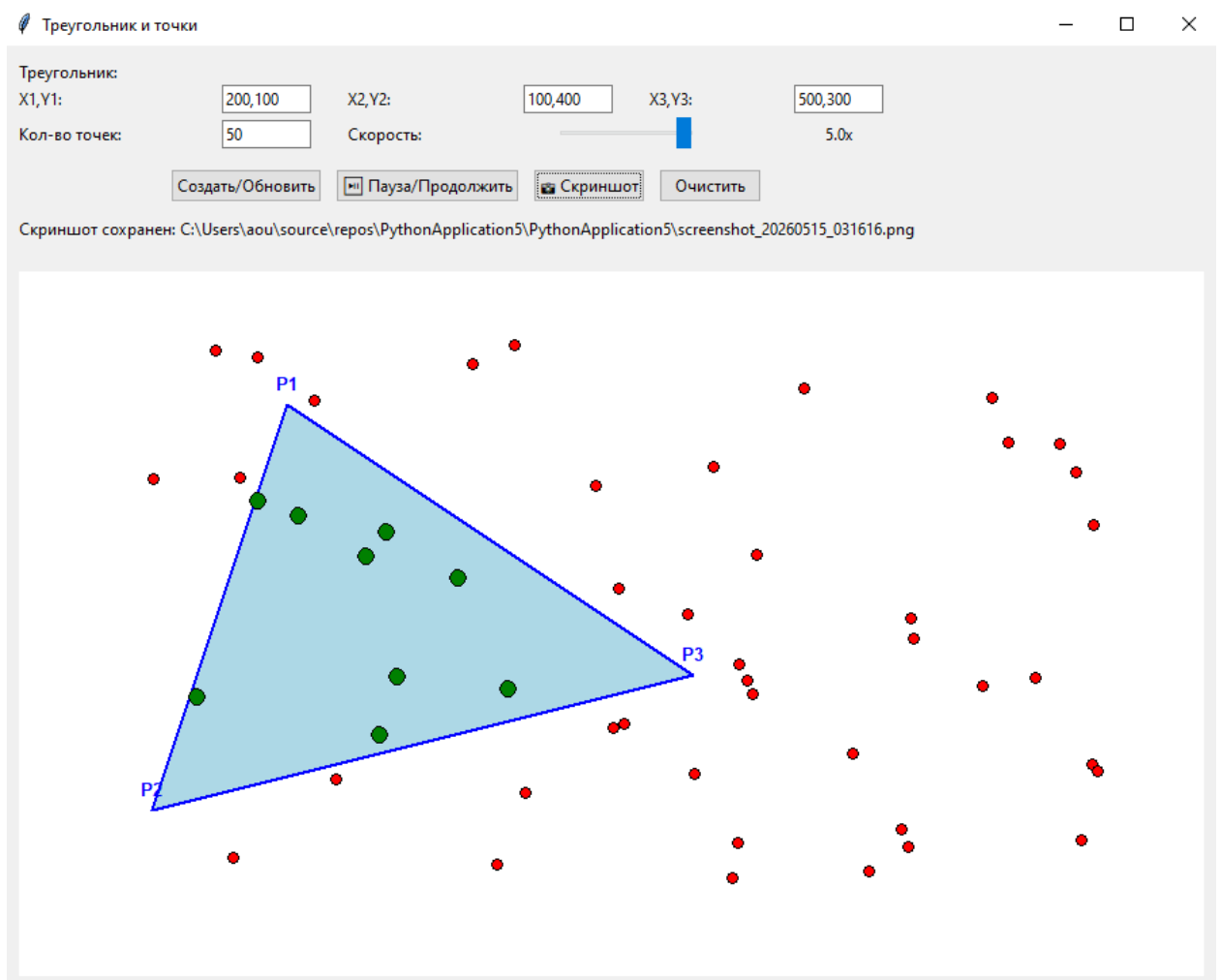
```
self.stats_label.configure(text=f"Скриншот сохранен: {os.path.abspath(filename)}")
```

```
if __name__ == "__main__":
```

```
    root = tk.Tk()
```

```
    app = App(root)
```

```
    root.mainloop()
```



Задание 2. Реализовать построение заданного типа фрактала по варианту

Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала

1) Множество Мальдеброт

```
import tkinter as tk
from tkinter import ttk, messagebox
import numpy as np
from PIL import Image, ImageTk, ImageGrab
import os
from datetime import datetime
import colorsys

class MandelbrotSet:
    def __init__(self, width=800, height=600, max_iter=100):
        self.width = width
        self.height = height
        self.max_iter = max_iter
        self.zoom = 1.0
        self.offset_x = -0.5
        self.offset_y = 0.0

    def generate(self):
        """Генерирует массив итераций для множества Мандельброта"""
        # Создаем сетку координат
        x = np.linspace(-2.5/self.zoom + self.offset_x,
                        1.5/self.zoom + self.offset_x, self.width)
        y = np.linspace(-2.0/self.zoom + self.offset_y,
                        2.0/self.zoom + self.offset_y, self.height)

        # Создаем матрицы
        X, Y = np.meshgrid(x, y)
```

```
C = X + 1j * Y
```

```
Z = np.zeros_like(C, dtype=np.complex128)
```

```
# Матрица для хранения количества итераций
```

```
iterations = np.zeros(C.shape, dtype=np.int32)
```

```
# Вычисляем множество Мандельброта
```

```
mask = np.ones(C.shape, dtype=bool)
```

```
for i in range(self.max_iter):
```

```
    Z[mask] = Z[mask]**2 + C[mask]
```

```
    mask = np.abs(Z) < 2
```

```
    iterations += mask
```

```
return iterations
```

```
def zoom_in(self, x, y, factor=2.0):
```

```
    """Приближение к указанной точке"""
```

```
    # Конвертируем координаты холста в комплексную плоскость
```

```
    complex_x = (x - self.width/2) / (self.width/4) / self.zoom + self.offset_x
```

```
    complex_y = (y - self.height/2) / (self.height/4) / self.zoom + self.offset_y
```

```
    self.zoom *= factor
```

```
    self.offset_x = complex_x
```

```
    self.offset_y = complex_y
```

```
def reset_view(self):
```

```
    """Сброс параметров отображения"""
```

```
    self.zoom = 1.0
```

```
    self.offset_x = -0.5
```

```
    self.offset_y = 0.0
```

```
class MandelbrotApp:
    def __init__(self, root):
        self.root = root

        self.root.title("Множество Мандельброта")
        self.root.geometry("1000x800")

        # Параметры фрактала
        self.width = 800
        self.height = 600
        self.max_iter = tk.IntVar(value=100)
        self.color_scheme = tk.StringVar(value="rainbow")
        self.zoom_factor = tk.DoubleVar(value=2.0)
        self.paused = False

        # Создаем объект фрактала
        self.mandelbrot = MandelbrotSet(self.width, self.height, self.max_iter.get())

        # Флаг для отмены вычислений
        self.computing = False

        self.setup_ui()

    def setup_ui(self):
        # Главный контейнер
        main_frame = ttk.Frame(self.root, padding=10)
        main_frame.pack(fill=tk.BOTH, expand=True)

        # Панель управления
        control_frame = ttk.LabelFrame(main_frame, text="Управление", padding=10)
        control_frame.pack(side=tk.TOP, fill=tk.X, pady=(0, 10))

        # Первая строка управления
```

```
row1 = ttk.Frame(control_frame)
```

```
row1.pack(fill=tk.X, pady=5)
```

```
ttk.Label(row1, text="Итерации:").pack(side=tk.LEFT, padx=5)
```

```
self.iter_entry = ttk.Entry(row1, textvariable=self.max_iter, width=10)
```

```
self.iter_entry.pack(side=tk.LEFT, padx=5)
```

```
ttk.Label(row1, text="Цветовая схема:").pack(side=tk.LEFT, padx=5)
```

```
color_combo = ttk.Combobox(row1, textvariable=self.color_scheme,  
                           values=["rainbow", "fire", "ocean", "grayscale", "neon"],  
                           width=15, state="readonly")
```

```
color_combo.pack(side=tk.LEFT, padx=5)
```

```
color_combo.set("rainbow")
```

```
ttk.Label(row1, text="Масштаб:").pack(side=tk.LEFT, padx=5)
```

```
self.zoom_label = ttk.Label(row1, text="1.0x")
```

```
self.zoom_label.pack(side=tk.LEFT, padx=5)
```

```
# Вторая строка управления
```

```
row2 = ttk.Frame(control_frame)
```

```
row2.pack(fill=tk.X, pady=5)
```

```
ttk.Label(row2, text="Фактор зума:").pack(side=tk.LEFT, padx=5)
```

```
zoom_scale = ttk.Scale(row2, from_=1.5, to=10.0, variable=self.zoom_factor,  
                       orient=tk.HORIZONTAL, length=150)
```

```
zoom_scale.pack(side=tk.LEFT, padx=5)
```

```
self.zoom_factor_label = ttk.Label(row2, text="2.0x")
```

```
self.zoom_factor_label.pack(side=tk.LEFT, padx=5)
```

```
self.zoom_factor.trace_add("write", lambda *args:
```

```
    self.zoom_factor_label.configure(  
        text=f'{self.zoom_factor.get():.1f}x'))
```

```

# Третья строка - кнопки

row3 = ttk.Frame(control_frame)

row3.pack(fill=tk.X, pady=5)

    ttk.Button(row3, text="🔄 Перерисовать", command=self.redraw).pack(side=tk.LEFT,
padx=5)

    ttk.Button(row3, text="⏸ Пауза", command=self.toggle_pause).pack(side=tk.LEFT,
padx=5)

    ttk.Button(row3, text="🔍 Сбросить вид", command=self.reset_view).pack(side=tk.LEFT,
padx=5)

    ttk.Button(row3, text="📷 Скриншот",
command=self.take_screenshot).pack(side=tk.LEFT, padx=5)

    ttk.Button(row3, text="💾 Сохранить изображение",
command=self.save_image).pack(side=tk.LEFT, padx=5)

# Информационная строка

self.info_label = ttk.Label(control_frame, text="Готов к работе. Кликните для
приближения")

self.info_label.pack(fill=tk.X, pady=5)

# Статистика

self.stats_label = ttk.Label(control_frame, text="")

self.stats_label.pack(fill=tk.X)

# Холст для отображения

self.canvas = tk.Canvas(main_frame, width=self.width, height=self.height,
bg='black', cursor="crosshair")

self.canvas.pack(expand=True)

# Привязка событий мыши

self.canvas.bind("<Button-1>", self.on_canvas_click)

self.canvas.bind("<B1-Motion>", self.on_canvas_drag)

self.canvas.bind("<ButtonRelease-1>", self.on_canvas_release)

```

```

# Координаты для перетаскивания
self.drag_start = None

# Генерируем начальный фрактал
self.root.after(100, self.redraw)

def get_color(self, iterations, max_iter):
    """Получение цвета в зависимости от схемы"""
    if iterations == max_iter:
        return (0, 0, 0) # Черный для точек множества

    ratio = iterations / max_iter

    if self.color_scheme.get() == "rainbow":
        hue = ratio * 0.7
        return tuple(int(255 * c) for c in colorsys.hsv_to_rgb(hue, 1.0, 1.0))

    elif self.color_scheme.get() == "fire":
        r = min(255, int(ratio * 500))
        g = min(255, int(ratio * 200))
        b = min(255, int(ratio * 50))
        return (r, g, b)

    elif self.color_scheme.get() == "ocean":
        r = min(255, int(ratio * 100))
        g = min(255, int(ratio * 200))
        b = min(255, int(ratio * 400))
        return (r, g, b)

    elif self.color_scheme.get() == "neon":
        hue = ratio * 0.8 + 0.6

```

```

        saturation = 0.8 + 0.2 * np.sin(ratio * 10)
        return tuple(int(255 * c) for c in colorsys.hsv_to_rgb(hue, saturation, 1.0))

    else: # grayscale
        v = int(255 * (1 - ratio))
        return (v, v, v)

def redraw(self):
    """Перерисовка фрактала"""
    if self.computing:
        return

    self.computing = True
    self.info_label.configure(text="Вычисление... Пожалуйста, подождите")
    self.root.update()

    try:
        # Обновляем параметры
        self.mandelbrot.max_iter = self.max_iter.get()

        # Генерируем фрактал
        iterations = self.mandelbrot.generate()

        # Создаем изображение
        image = Image.new('RGB', (self.width, self.height))
        pixels = image.load()

        for y in range(self.height):
            for x in range(self.width):
                color = self.get_color(iterations[y, x], self.max_iter.get())
                pixels[x, y] = color

```



```

# Отображаем на холсте
self.photo = ImageTk.PhotoImage(image)
self.canvas.delete("all")
self.canvas.create_image(self.width/2, self.height/2, image=self.photo)

# Обновляем статистику
points_in_set = np.sum(iterations == self.max_iter.get())
total_points = self.width * self.height
percentage = (points_in_set / total_points) * 100

self.stats_label.configure(
    text=f"Точек в множестве: {points_in_set} ({percentage:.1f}%) | "
        f"Масштаб: {self.mandelbrot.zoom:.2f}x | "
        f"Смещение: ({self.mandelbrot.offset_x:.4f}, {self.mandelbrot.offset_y:.4f})"
)

self.zoom_label.configure(text=f"{self.mandelbrot.zoom:.2f}x")

except Exception as e:
    messagebox.showerror("Ошибка", f"Ошибка при вычислении: {str(e)}")
finally:
    self.computing = False
    self.info_label.configure(text="Готово. Кликните для приближения")

def on_canvas_click(self, event):
    """Обработка клика для зума"""
    if not self.paused:
        self.mandelbrot.zoom_in(event.x, event.y, self.zoom_factor.get())
        self.redraw()

def on_canvas_drag(self, event):
    """Начало перетаскивания"""

```

```

if self.drag_start is None:
    self.drag_start = (event.x, event.y)

def on_canvas_release(self, event):
    """Конец перетаскивания"""
    if self.drag_start:
        dx = event.x - self.drag_start[0]
        dy = event.y - self.drag_start[1]

        if abs(dx) > 5 or abs(dy) > 5: # Минимальное расстояние для перетаскивания
            # Перемещение вида
            scale = 4.0 / (self.width * self.mandelbrot.zoom)
            self.mandelbrot.offset_x -= dx * scale
            self.mandelbrot.offset_y -= dy * scale
            self.redraw()

        self.drag_start = None

def toggle_pause(self):
    """Пауза/продолжение"""
    self.paused = not self.paused
    if self.paused:
        self.info_label.configure(text="ПАУЗА - интерактивность отключена")
    else:
        self.info_label.configure(text="Работа возобновлена")

def reset_view(self):
    """Сброс вида"""
    self.mandelbrot.reset_view()
    self.max_iter.set(100)
    self.redraw()

```

```

def take_screenshot(self):
    """Создание скриншота"""
    try:
        x = self.root.winfo_rootx() + self.canvas.winfo_x() + 10
        y = self.root.winfo_rooty() + self.canvas.winfo_y() + 10
        width = self.canvas.winfo_width()
        height = self.canvas.winfo_height()

        screenshot = ImageGrab.grab(bbox=(x, y, x + width, y + height))

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = f"mandelbrot_screenshot_{timestamp}.png"
        screenshot.save(filename)

        self.info_label.configure(text=f"Скриншот сохранен: {os.path.abspath(filename)}")
    except Exception as e:
        messagebox.showerror("Ошибка", f"Не удалось создать скриншот: {str(e)}")

def save_image(self):
    """Сохранение изображения в высоком качестве"""
    try:
        # Создаем изображение в высоком разрешении
        hd_width, hd_height = self.width * 2, self.height * 2

        mandelbrot_hd = MandelbrotSet(hd_width, hd_height, self.max_iter.get())
        mandelbrot_hd.zoom = self.mandelbrot.zoom
        mandelbrot_hd.offset_x = self.mandelbrot.offset_x
        mandelbrot_hd.offset_y = self.mandelbrot.offset_y

        self.info_label.configure(text="Сохранение в HD качестве...")
        self.root.update()

```

```

iterations = mandelbrot_hd.generate()

image = Image.new('RGB', (hd_width, hd_height))
pixels = image.load()

for y in range(hd_height):
    for x in range(hd_width):
        color = self.get_color(iterations[y, x], self.max_iter.get())
        pixels[x, y] = color

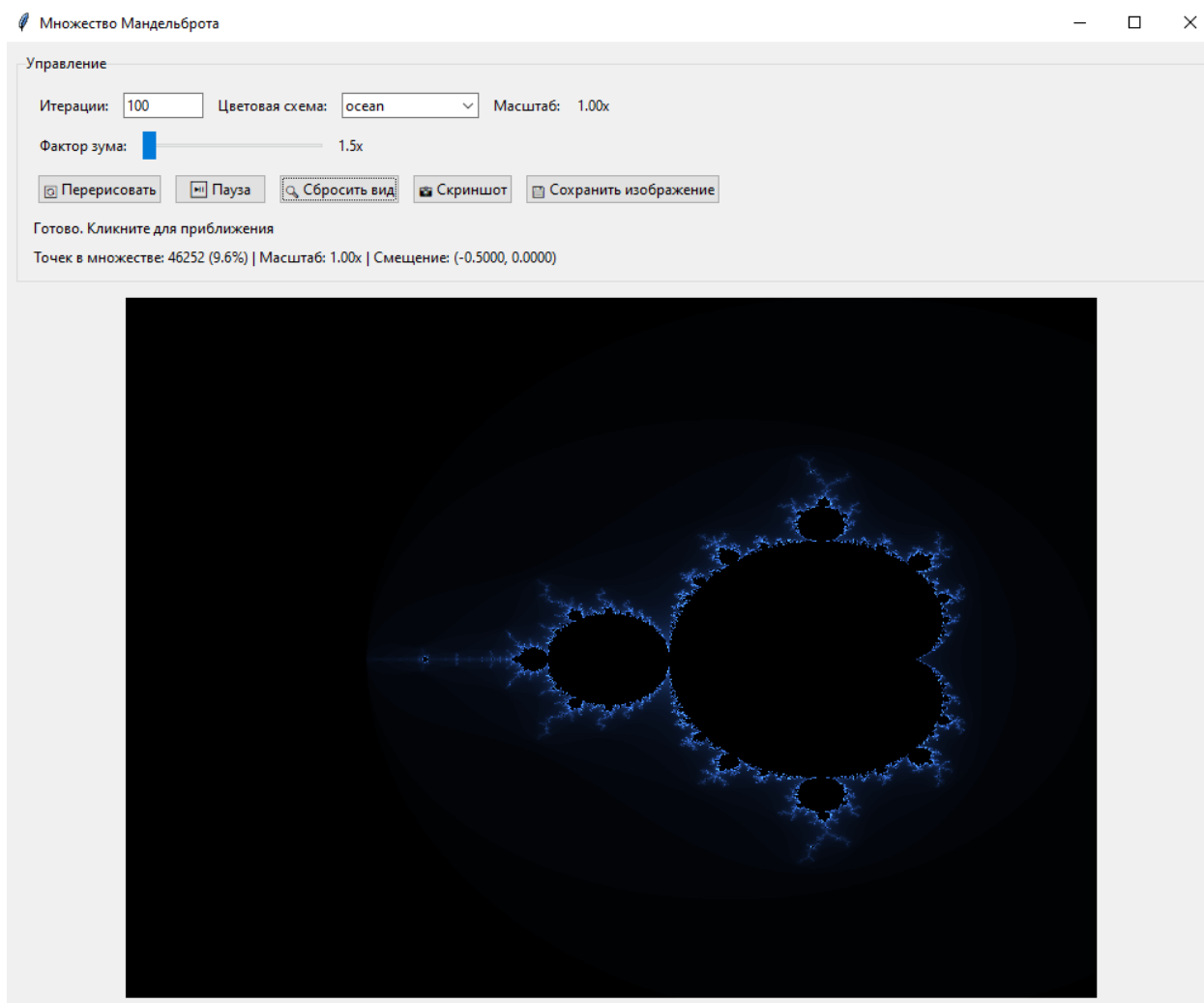
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
filename = f"mandelbrot_hd_{timestamp}.png"
image.save(filename, quality=95)

self.info_label.configure(
    text=f"HD изображение сохранено: {os.path.abspath(filename)}")

except Exception as e:
    messagebox.showerror("Ошибка", f"Не удалось сохранить изображение: {str(e)}")

if __name__ == "__main__":
    root = tk.Tk()
    app = MandelbrotApp(root)
    root.mainloop()

```



Вывод: были отработаны базовые знания языка программирования Python при решении практических задач